학습 노트: FastAPI 엔드포인트 설계 (라우터 + 202 + BackgroundTasks + Mock 테스트)

작성일: 2026-05-24 관련 파일: `dev2/api/scan.py`, `dev2/api/insights.py`, `dev2/api/issues.py`, `dev2/main.py`, `dev2/db.py`, `dev2/tests/test_insights.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 6 — FastAPI 엔드포인트 (POST /scan + Insights API 3종 + PATCH /issues/{id}/dismiss)

한 줄 요약

"PR을 분석해줘" 같은 요청을 받는 창구(웹 **API**)를 만드는 일입니다. 오래 걸리는 일(LLM 분석)은 접수증만 주고 뒤에서 처리하고, 통계 조회는 바로 답을 주며, 실제 DB·LLM 없이도 테스트가 통과하도록 만들었습니다.

이 노트에서 다루는 10가지 개념

1. FastAPI APIRouter — 창구를 파일별로 나누기
2. HTTP 상태코드 202 vs 200 — "접수했어요" vs "다 됐어요"
3. BackgroundTasks — 응답 먼저 주고 뒤에서 일하기
4. HTTP 동사 (GET / POST / PATCH) — 요청에 "성격" 부여하기
5. SQLAlchemy AsyncSession — DB와 대화하는 안전한 통로
6. Lazy Import — 필요할 때만 짐 꺼내기
7. db.py Lazy 초기화 — DB 엔진을 최초 사용 순간에만 켜기
8. Mock으로 async with 흉내내기 — DB 없이 테스트하기
9. "patch where it's used" — Mock을 어디에 걸어야 하나
10. ASGITransport — 진짜 서버 없이 API 테스트하기

개념 1. FastAPI APIRouter — 창구를 파일별로 나누기

🏬 비유

큰 시청에 가면 창구가 "여권 발급", "전입신고", "민원 접수"로 나뉘어 있습니다. 한 창구에서 모든 걸 다 처리하면 줄이 엉키고, 직원도 헷갈립니다. APIRouter는 이 "창구를 종류별로 나누는" 도구입니다. 그리고 `include_router`는 나눠 만든 창구들을 한 건물(앱)에 입주시키는 것입니다.

📚 개념

`APIRouter`는 관련된 엔드포인트(URL)들을 한 묶음으로 모아 별도 파일로 분리하는 FastAPI 객체다. `prefix`로 공통 경로를, `tags`로 문서 그룹명을 지정한다. 메인 앱에서 `app.include_router(router)`로 등록하면 그 라우터의 모든 경로가 앱에 합쳐진다.

```
# dev2/api/insights.py
router = APIRouter(prefix="/insights", tags=["insights"])

@router.get("/team")          # 실제 경로는 prefix + 경로 = "/insights/team"
async def get_team_insights():
    ...
# dev2/main.py — 조립만 담당
app = FastAPI(...)
app.include_router(scan_router)      # POST /scan
app.include_router(insights_router)  # GET /insights/*  (prefix="/insights")
app.include_router(issues_router)    # PATCH /issues/*  (prefix="/issues")
```

🤔 왜 쓰나

- 관심사 분리: `scan`(트리거), `insights`(통계), `issues`(오탐 표시)는 성격이 완전히 다릅니다. 한 파일에 몰아넣으면 수백 줄짜리 거대 파일이 됩니다.
- 병렬 개발 (이 프로젝트의 핵심 이유): 6명이 동시에 개발하는데(CLAUDE.md §15), 모두 `main.py` 하나를 고치면 Git 충돌(merge conflict)이 끊임없이 납니다. 파일을 나누면 각자 자기 파일만 건드립니다.
- 테스트 독립성: 라우터별로 Mock을 따로 걸어 격리 테스트할 수 있습니다.

핵심: `main.py`는 "조립"만 합니다. 비즈니스 로직(SQL, 분기)은 각 라우터 파일에 둡니다. 이게 CLAUDE.md의 "FastAPI 라우터는 수신·직렬화만 담당" 원칙과 같은 맥락입니다.

개념 2. HTTP 상태코드 202 vs 200 — "접수했어요" vs "다 됐어요"

🍕 비유

피자를 주문할 때 두 가지 경우가 있습니다. - 편의점에서 음료수 사기: 돈 내면 그 자리에서 바로 음료를 받습니다 → **200 OK** (요청 = 완료) - 피자 배달 주문: "주문 접수됐습니다, 30분 뒤 도착합니다" 하고 영수증(접수증)만 받습니다. 피자는 아직 안 왔습니다 → **202 Accepted** (요청은 받았지만 결과는 나중)

📚 개념

- **200 OK**: 요청을 처리해서 결과를 지금 응답에 담아 보냈다.
- **202 Accepted**: 요청을 접수만 했고, 실제 처리는 나중에 진행된다. 응답에는 결과가 아니라 "접수증"이 담긴다.

```
# dev2/api/scan.py
@router.post("/scan", status_code=202)   # ← 202로 명시
async def trigger_scan(payload, background_tasks):
    ...
    return {"status": "accepted", "task_id": payload.task_id}  # 결과 아님, 접수증
```

🤔 왜 쓰나 (면접 단골 질문)

LLM 분석은 수 초 ~ 수십 초가 걸립니다. 만약 200으로 "분석 다 끝낼 때까지" 클라이언트를 붙잡아 두면: - 클라이언트(브라우저/GitHub Webhook)가 **HTTP** 타임아웃(보통 30초)에 걸려 연결이 끊깁니다. - 끊기면 분석은 했는데 결과를 못 주는 황당한 상황이 됩니다.

그래서 **"일단 접수증(202)부터 빠르게 주고, 분석은 뒤에서"** 하는 구조로 분리합니다. 결과는 나중에 별도 경로(GitHub 코멘트, Slack 알림, 또는 GET 조회)로 전달됩니다.

> 면접 답변 한 줄: "오래 걸리는 비동기 작업은 200으로 끝까지 기다리게 하면 타임아웃 위험이 있어, 202로 즉시 접수증을 반환하고 실제 처리는 백그라운드로 분리합니다."

개념 3. BackgroundTasks — 응답 먼저 주고 뒤에서 일하기

🔍 비유

식당 종업원(서버)이 주문(요청)을 받습니다. 손님에게 "주문 들어갔습니다"라고 먼저 말하고(응답 전송), 그 다음에 주방에 "이거 만들어주세요" 하고 메모를 붙여 둡니다(백그라운드 작업 등록). 손님은 기다리지 않고 자리에 앉고, 요리는 뒤에서 만들어집니다. 개념 2의 202와 짝을 이루는 "실제 일하는 부분"이 바로 이것입니다.

📚 개념

> `BackgroundTasks`는 FastAPI가 제공하는 내장 기능으로, **HTTP** 응답을 클라이언트에 보낸 직후 등록된 함수를 실행한다. `background_tasks.add_task(함수, 인자)`로 등록만 하면, 함수 호출이 그 자리에서 일어나지 않고 응답 이후로 미뤄진다.

```
# dev2/api/scan.py (실제 코드)
@router.post("/scan", status_code=202)
async def trigger_scan(payload: TaskPayload, background_tasks: BackgroundTasks):
```

```
    initial_state = _payload_to_state(payload)        # 1. 입력을 State로 변환
    background_tasks.add_task(run_dev2, initial_state) # 2. 등록만! 아직 실행 안 함
    return {"status": "accepted", "task_id": payload.task_id}  # 3. 즉시 응답
    # 4. (응답 전송 후) run_dev2(initial_state)가 백그라운드에서 실행됨
```

실행 순서가 핵심입니다: 1. `_payload_to_state()` — 입력 검증·변환 (지금 실행) 2. `add_task()` — "이거 나중에 실행해줘" 등록 (실행 아님, 예약) 3. `return` — 202 접수증을 클라이언트로 전송 4. 응답이 나간 뒤 `run_dev2()`가 LangGraph 5노드 파이프라인을 돌림

🤔 왜 쓰나 / 언제 쓰나

- 적합: 이메일 발송, 알림, 로깅, 그리고 이 프로젝트처럼 응답을 막으면 안 되는 무거운 후처리.
- 부적합: 몇 시간씩 걸리거나 실패 시 재시도·추적이 꼭 필요한 작업. 이런 건 Celery, Redis Queue, Airflow 같은 전용 작업 큐를 씁니다 (서버가 죽으면 BackgroundTasks는 그냥 사라지기 때문).
- 우리는 "수십 초짜리 단발성 분석"이라 BackgroundTasks가 딱 맞습니다. (CLAUDE.md §6의 타임아웃 분리 원칙과도 연결됩니다.)

개념 4. HTTP 동사 (GET / POST / PATCH) — 요청에 "성격" 부여하기

📮 비유

우체국에 갈 때 행동의 종류가 다릅니다. - **GET**: 우편함을 "열어보기만" 하기 (꺼내지도, 바꾸지도 않음) → 몇 번 봐도 안전 - **POST**: 새 편지를 "부치기" (보낼 때마다 새 편지가 생김) → 두 번 누르면 편지 두 통 - **PATCH**: 봉투에 적힌 주소 중 "한 줄만 고쳐 쓰기" → 같은 수정을 몇 번 해도 결과는 똑같음

📚 개념

> HTTP 동사(메서드)는 요청이 서버에 어떤 영향을 주는지를 약속한 규칙이다. - **GET**: 읽기 전용. 서버 상태를 바꾸지 않는다(side effect 없음). 멱등**(idempotent)**. - **POST**: 새 자원/작업 생성. 호출할 때마다 새 결과 → 비멱등. - **PATCH**: 자원의 일부 필드만 수정. 보통 멱등. - (참고) PUT: 자원 전체 교체. DELETE: 삭제.
>
> 멱등**(idempotent)** = "같은 요청을 여러 번 보내도 결과가 똑같다." 네트워크가 끊겨 재시도해도 안전한지를 판단하는 핵심 개념.

이 프로젝트의 선택 근거:

엔드포인트	동사	이유
`/insights/developer/{id}`, `/insights/team`, `/insights/trends`	**GET**	SELECT만 함, 데이터 안 바꿈 → 읽기 전용
`/scan`	**POST**	새 분석 "작업"을 생성, 부를 때마다 새 작업 → 비멱등
`/issues/{id}/dismiss`	**PATCH**	`is_false_positive` 한 필드만 true로 변경, 여러 번 해도 결과 동일 → 멱등

🧐 왜 **PATCH**인가 **(DELETE/PUT**이 아닌 이유**)**

- **DELETE**가 아닌 이유: 오탐 표시는 이슈를 "지우는" 게 아닙니다. 기록은 남겨야 합니다 (감사 추적, Dev5 피드백 학습용). 그래서 "표시만" 하는 PATCH.
- **PUT**이 아닌 이유: PUT은 레코드 전체를 통째로 교체합니다. 우리는 `is_false_positive` 딱 한 칸만 바꾸므로 PATCH가 정확합니다.

개념 5. SQLAlchemy AsyncSession — DB와 대화하는 안전한 통로

☎️ 비유

DB에 질문하려면 "전화 회선(세션)"을 하나 빌려야 합니다. 다 쓰고 나면 회선을 반드시 반납해야 다른 사람이 씁니다. `async with`는 "통화 끝나면 자동으로 전화를 끊어주는" 똑똑한 전화기입니다. 깜빡하고 안 끊을 일이 없습니다.

📚 개념

- **AsyncSession**: 비동기로 DB와 대화하는 세션 객체. `await`로 쿼리를 보내고 기다린다.
- 컨텍스트 매니저 **(`async with`)**: 블록을 벗어날 때 세션을 자동으로 닫아(반환) 준다. 누수(leak) 방지.
- 파라미터 바인딩 **(`:param`)**: 값을 SQL 문자열에 직접 끼워 넣지 않고 따로 전달 → SQL 인젝션 차단.
- **`r.mappings().all()`**: 쿼리 결과를 `{컬럼명: 값}` 딕셔너리 목록으로 받는다. (그냥 `.all()`은 튜플이라 컬럼명을 모름)

```
# dev2/api/insights.py
async with AsyncSessionLocal() as s:        # 회선 빌리기 (블록 끝나면 자동 반납)
    r = await s.execute(
```

```
        text("""
            SELECT si.rule_id, si.severity, COUNT(*) as count
            FROM scan_issues si
            JOIN scan_executions se ON si.scan_id = se.id
            WHERE se.developer_id = :did     -- ← :did 자리표시자
            GROUP BY si.rule_id, si.severity
            ORDER BY count DESC
        """),
        {"did": developer_id},               # ← 값은 따로 전달 (바인딩)
    )
    issues = [dict(row) for row in r.mappings().all()]  # 컬럼명 dict 목록
```

🧐 왜 파라미터 바인딩이 필수인가 (보안)

만약 `f"... WHERE developer_id = '{developer_id}'"` 처럼 f-string으로 값을 끼우면, 누가 `developer_id`에 `'; DROP TABLE scan_issues; --` 같은 값을 넣었을 때 테이블이 통째로 삭제될 수 있습니다(SQL 인젝션). 바인딩(`:did`)을 쓰면 DB가 그 값을 "그냥 문자열 데이터"로만 취급해서 명령으로 실행되지 않습니다. CLAUDE.md §6의 "f-string으로 SQL 조합 금지" 규칙이 이것입니다.

개념 6. Lazy Import — 필요할 때만 짐 꺼내기

👛 비유

캠핑을 갈 때 텐트를 "차에 미리 다 펼쳐서" 가면 무겁고, 캠핑장 안 가면 헛수고입니다. 보통은 차 트렁크에 접어 넣고, 캠핑장에 도착한 순간(필요한 그 시점) 펼칩니다. Lazy import는 모듈을 파일 맨 위에서 미리 불러오지 않고, 함수가 실제로 호출될 때 불러오는 것입니다.

🧦 개념

> 보통 import는 파일 최상단에 둬서 파일을 읽는 순간 함께 로드된다. **Lazy import**는 import 문을 함수 내부에 넣어, 그 함수가 호출되는 순간에만 모듈을 로드한다.

```
# dev2/api/insights.py
@router.get("/developer/{developer_id}")
async def get_developer_insights(developer_id: str):
    from dev2.db import AsyncSessionLocal   # ← 함수 내부에서 import (lazy)
    async with AsyncSessionLocal() as s:
        ...
```

🧐 왜 쓰나 (이 프로젝트의 진짜 이유)

`dev2.db` 모듈은 `asyncpg`(PostgreSQL 비동기 드라이버)를 필요로 합니다. 만약 `insights.py` 맨 위에서 `from dev2.db import AsyncSessionLocal`을 하면: - **CI**/테스트 환경에 **asyncpg**가

안 깔려 있으면 `insights.py`를 import하는 순간 에러가 터집니다. - 그러면 DB랑 전혀 상관 없는 테스트(예: 라우터 등록 확인)까지 전부 실패합니다.

함수 안으로 옮기면, 그 함수를 실제로 호출할 때만 asyncpg가 필요해집니다. 테스트에서는 Mock으로 갈아끼우므로 asyncpg가 없어도 import·테스트가 통과합니다. (CLAUDE.md §13: "실제 DB/LLM 없이 통과"의 기반)

> 주의: lazy import는 "이 모듈은 무거운/선택적 의존성을 쓴다"는 신호일 때 적절합니다. 평범한 가벼운 모듈은 그냥 상단 import가 정석입니다. (그래서 코드에 `# noqa: PLC0415` 주석으로 "의도된 lazy import"임을 명시했습니다.)

개념 7. db.py Lazy 초기화 — DB 엔진을 최초 사용 순간에만 켜기

🕯️ 비유

사무실에 비싼 복합기(DB 엔진)가 있습니다. 출근하자마자 켜두면, 아무도 안 쓰는 날엔 전기만 낭비됩니다. 똑똑한 방법은 누군가 처음 인쇄 버튼을 누르는 순간 켜고, 그 다음부터는 켜진 채로 계속 쓰는 것입니다. 한 번만 켜고 계속 재사용 → 이게 싱글톤(singleton)에 가까운 발상입니다.

📚 개념

> 개념 6의 lazy import가 "모듈 가져오기를 미루는 것"이라면, 이건 한 단계 더 나아가 무거운 객체(**DB** 엔진) 생성 자체를 최초 사용 시점까지 미루는 패턴이다. 전역 변수를 `None`으로 두고, 처음 필요할 때 만든 뒤 재사용한다.

```python
# dev2/db.py
_engine = None
_session_factory = None

def _get_session_factory():
    global _engine, _session_factory
    if _session_factory is None:                          # 최초 1회만 진입
        _engine = create_async_engine(DATABASE_URL)    # ← 여기서 asyncpg 필요
        _session_factory = sessionmaker(
            _engine, class_=AsyncSession, expire_on_commit=False
        )
    return _session_factory                               # 이후엔 기존 것 재사용

def AsyncSessionLocal():
    return _get_session_factory()()   # 세션 컨텍스트 매니저 반환
```

🧐 왜 쓰나

- **import** 안전성: 모듈 최상단에서 `create_async_engine(...)`을 호출하면 `dev2.db`를 import 하는 순간 asyncpg가 없으면 폭발합니다. 함수 안으로 넣으면 import는 안전하고, 실제 DB 쓸 때만 asyncpg가 필요합니다 (개념 6과 같은 목표를 객체 생성 레벨에서 달성).
- 자원 절약: 엔진은 DB 커넥션 풀을 가진 무거운 객체입니다. 매 요청마다 만들면 낭비 → `if _session_factory is None`으로 딱 한 번만 만들고 재사용합니다.

개념 8. Mock으로 async with 흉내내기 — DB 없이 테스트하기

🎭 비유

영화에서 배우가 실제로 절벽에서 떨어지지 않습니다. 스턴트 대역**(Mock)**이 대신합니다. 관객(테스트)이 보기엔 똑같이 동작하지만, 진짜 위험(진짜 DB 연결)은 없습니다. 우리는 `AsyncSessionLocal()`이라는 "진짜 DB 통로"를 가짜 대역으로 바꿔치기합니다.

🧩 개념

`async with AsyncSessionLocal() as s:` 한 줄이 동작하려면 가짜 객체가 갖춰야 할 게 여러 겹이다: 1. `AsyncSessionLocal()` 호출 → 컨텍스트 매니저를 반환해야 함 2. 그 매니저는 `__aenter__`/`__aexit__`(async 버전)을 지원해야 함 → `AsyncMock` 3. `__aenter__`이 세션(s)을 내놔야 함 4. `await s.execute(...)`가 결과 객체를 내놔야 함 5. 결과`.mappings().all()`이 행 목록을 내놔야 함

```
# dev2/tests/test_insights.py
def _mock_db_ctx(rows=None):
    rows = rows or []

    # (5) r.mappings().all() → rows  를 흉내
    mock_result = MagicMock()
    mock_result.mappings.return_value.all.return_value = rows

    # (4) await s.execute(...) → mock_result  를 흉내 (await 가능해야 하므로
AsyncMock)
    mock_session = AsyncMock()
    mock_session.execute.return_value = mock_result

    # (1~3) async with ... as s → mock_session  을 흉내
    mock_cm = AsyncMock()
    mock_cm.__aenter__.return_value = mock_session  # as s 자리
    mock_cm.__aexit__.return_value = False          # 예외 전파 허용
    return mock_cm
```

🧐 핵심 포인트: **MagicMock vs AsyncMock**

- **AsyncMock**: `await` 붙여 호출하는 것들 (`s.execute`, `__aenter__`, `__aexit__`). 비동기 흉내.
- **MagicMock**: `await` 없이 그냥 호출하는 것들 (`mappings()`, `all()`). 동기 흉내.
- `mock.xxx.return_value.yyy.return_value` 체인은 "`mock.xxx()` 결과에 또 `.yyy()`를 부른 결과"를 미리 정해두는 문법입니다. `r.mappings().all()`처럼 점이 이어지는 호출을 흉내 낼 때 씁니다.

왜 이렇게까지 하나

실제 PostgreSQL을 띄우지 않고도 "엔드포인트가 DB 결과를 받아 올바른 JSON으로 포장하는가"를 검증할 수 있습니다. 테스트가 빠르고(네트워크 없음), 어디서나 돌고(DB 설치 불필요), DB 상태에 흔들리지 않습니다(결정적).

개념 9. "patch where it's used" — Mock을 어디에 걸어야 하나

🚪 비유

같은 "철수"라도 회사에서 부르는 이름과 집에서 부르는 이름이 다릅니다. Mock을 걸 때는 "코드가 그 객체를 실제로 부르는 이름"에 걸어야 합니다. 엉뚱한 이름에 걸면 가짜로 안 바뀝니다.

🧩 개념

> Mock 패치의 원칙: "정의된 곳"이 아니라 "사용되는 곳"의 이름을 패치한다. 단, 그 "사용되는 이름"은 import 방식에 따라 달라진다.

이 프로젝트에 두 패턴이 공존합니다:

(A) 모듈 상단 **import** → 사용처 이름으로 패치

```
# scan.py 상단: from dev2.graph import run_dev2
# → scan.py 안에 run_dev2라는 "복사된 이름"이 생김
with patch("dev2.api.scan.run_dev2", ...):   # ← 사용처(scan) 이름으로 패치
```

(B) 함수 내부 **lazy import** → 원본 모듈 이름으로 패치

```
# insights.py: 함수 안에서 from dev2.db import AsyncSessionLocal
# → 함수가 호출될 때마다 dev2.db에서 매번 새로 가져옴
with patch("dev2.db.AsyncSessionLocal", ...):  # ← 원본(dev2.db) 이름으로 패치
```

🧐 왜 패턴 **(B)**는 원본 위치인가

패턴 A는 파일을 로드할 때 `run_dev2`라는 이름을 `scan.py` 네임스페이스에 복사해 둡니다. 그래서 그 복사본(`dev2.api.scan.run_dev2`)을 바꿔야 합니다.

패턴 B(lazy import)는 이름을 미리 복사하지 않습니다. 함수가 호출되는 매 순간 `dev2.db`로 가서 `AsyncSessionLocal`을 꺼냅니다. 그래서 복사본이 없고, 원본 **(`dev2.db.AsyncSessionLocal`)**을 바꾸면 호출 시점에 자동으로 가짜를 가져옵니다. (개념 6 lazy import의 부수 효과 — 패치가 더 단순해집니다.)

외우는 법: "상단 import면 그 파일 이름으로, 함수 안 import면 원본 모듈 이름으로 패치."

개념 **10. ASGITransport** — 진짜 서버 없이 **API** 테스트하기

🏠 비유

집을 사기 전에 "모델하우스"에서 구조를 둘러봅니다. 실제 집을 짓지 않아도 방 배치를 확인할 수 있죠. ASGITransport는 진짜 서버를 띄우지(포트 열고 네트워크 듣고) 않고, FastAPI 앱을 같은 프로세스 안에서 직접 호출해 테스트하게 해줍니다.

📚 개념

> **ASGI**는 파이썬 비동기 웹 앱의 표준 인터페이스다. `ASGITransport(app=app)`는 HTTP 소켓(네트워크) 없이 httpx 클라이언트의 요청을 곧장 FastAPI 앱 함수로 전달한다(in-process). `base_url`은 형식상 필요한 가짜 주소다.

```
# dev2/tests/test_insights.py
transport = ASGITransport(app=app)                                  # 앱을 직접 연결
async with AsyncClient(transport=transport, base_url="http://test") as client:
    r = await client.post("/scan", json={...})
assert r.status_code == 202
```

🧐 왜 이 방식인가 **(**옛 방식이 **deprecated** 된 이유**)**

- 이전 방식: `AsyncClient(app=app, ...)` — httpx 0.20부터 deprecated 되었습니다.
- httpx가 "transport(전송 방식)"라는 개념을 명시적으로 분리했기 때문입니다. `app=app`은 "이 앱으로 보낸다"는 의도가 숨어 있었지만, `transport=ASGITransport(app=app)`는 "ASGI 방식으로 이 앱에 직접 전달한다"가 명확합니다. 미래에 다른 transport(실제 네트워크, WSGI 등)로 바꿀 때 일관성이 생깁니다.
- 장점: 포트 충돌 없음, 빠름(네트워크 왕복 없음), 어디서나 동작.

🔁 전체 흐름 한눈에 보기

```
[POST /scan — 쓰기/트리거]
클라이언트 → POST /scan
   → FastAPI가 TaskPayload(Pydantic)로 자동 검증
   → _payload_to_state() 변환
```

```
   → background_tasks.add_task(run_dev2, state)   (예약만)
   → 202 {"status":"accepted", ...} 즉시 반환  ← 여기서 클라이언트는 해방
   ──(응답 전송 후)──
   → run_dev2(state) 백그라운드 실행 (LangGraph 5노드)

[GET /insights/* — 읽기/조회]
클라이언트 → GET /insights/team
   → (함수 내부) from dev2.db import AsyncSessionLocal   (lazy)
   → async with AsyncSessionLocal() as s:  (필요 시 엔진 lazy 초기화)
   → SELECT ... :param 바인딩 → r.mappings().all()
   → 200 {"top_issues":[...]} 반환

[PATCH /issues/{id}/dismiss — 일부 수정/멱등]
클라이언트 → PATCH /issues/1/dismiss
   → UPDATE scan_issues SET is_false_positive=true WHERE id=:id
   → 200 {"issue_id":1, "is_false_positive":true}
```

3rd-Project-sub2 프로젝트에서

파일	역할
`dev2/main.py`	FastAPI 앱 조립. 3개 라우터 + /health 등록 (비즈니스 로직 없음)
`dev2/api/scan.py`	POST /scan → 202 + BackgroundTasks로 `run_dev2` 비동기 실행
`dev2/api/insights.py`	GET /insights/{developer,team,trends} → 집계 통계 (읽기 전용, DB 실패 시 빈 결과)
`dev2/api/issues.py`	PATCH /issues/{id}/dismiss → 오탐 마킹 (멱등, 기록 유지)
`dev2/db.py`	AsyncSession lazy 초기화. `AsyncSessionLocal()` 컨텍스트 매니저 제공
`dev2/tests/test_insights.py`	5개 엔드포인트를 Mock(DB/run_dev2)으로 격리 테스트, ASGITransport 사용

이 엔드포인트들은 CLAUDE.md의 **CQRS** 분리(쓰기 = POST /scan, 읽기 = GET /insights), **§6** 보안(파라미터 바인딩), **§13 Mock** 테스트 원칙을 구현한 결과물입니다.

연관 개념

- **Pydantic** 검증 → `TaskPayload`로 요청 바디를 자동 검증 (CLAUDE.md §5-B 입력 계약)
- **LangGraph** 파이프라인 → `run_dev2`가 실행하는 5노드 그래프 (agent/langgraph.md)
- **asyncio** 병렬 실행 → 백그라운드 안에서 도구들을 동시 실행 (ai/asyncio_parallel.md)
- **UPSERT** → insights가 읽는 `scan_summaries`를 채우는 쓰기 패턴 (database/upsert.md)

복습 퀴즈 (5문항)

1. (이해) 친구가 "POST /scan에 왜 200이 아니라 202를 썼어?"라고 물으면 피자 배달 비유로 30초 안에 설명해보세요.
2. (이해) `BackgroundTasks`로 등록한 `run_dev2`는 응답을 보내기 전에 실행될까요, 후에 실행될까요? 그 이유는?
3. (적용) 새 엔드포인트 `DELETE /issues/{id}`를 추가한다면 적절할까요? 우리가 PATCH를 고른 이유를 근거로 판단해보세요.
4. (적용) `insights.py`의 SQL을 `f"... WHERE developer_id = '{developer_id}'"`로 바꾸면 어떤 보안 문제가 생기나요? 한 문장으로.
5. (비교) `patch("dev2.api.scan.run_dev2")`와 `patch("dev2.db.AsyncSessionLocal")`는 패치 위치가 다릅니다. 각각 왜 그 위치를 골랐나요? (힌트: import가 파일 상단인지 함수 내부인지)

정답이 막히면 개념 2·3(2번), 개념 4(3번), 개념 5(4번), 개념 6·9(5번)를 다시 보세요.